

AI Retail Insights Assistant

Technical work sample — submitted by Maurishi Singh for the Singapore AI Safety Fellowship

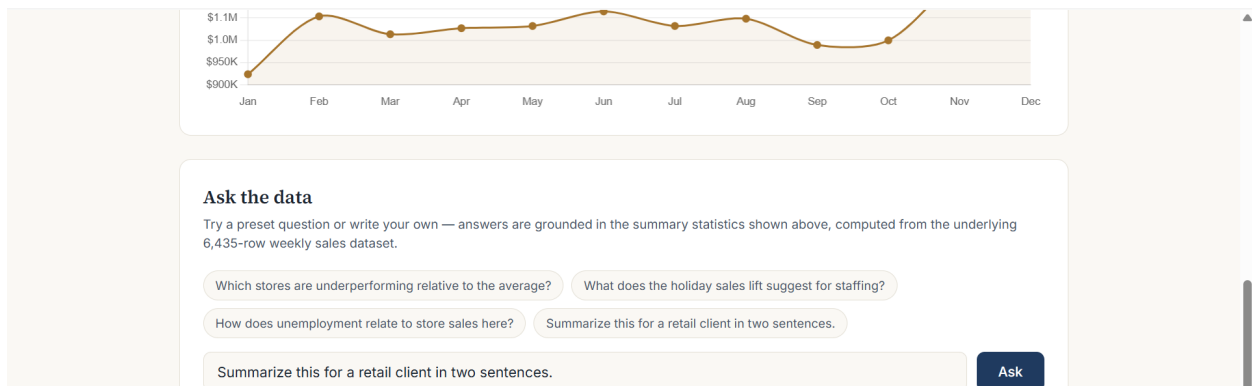
Links

Live demo: <https://maurishi09.github.io/retail-ai-insights/>

GitHub repository: <https://github.com/maurishi09/retail-ai-insights>

What this is

A small AI-powered tool built on a Walmart weekly sales dataset (45 stores, Feb 2010–Oct 2012). Instead of reading a static dashboard, a business user can ask a plain-English question — "which stores are underperforming?", "what does the holiday sales lift suggest for staffing?" — and get an answer grounded in the actual aggregated data, generated by an LLM constrained to that data.



The live app: sales trend chart, preset business questions, and a free-text query box.

Why I'm submitting this as my work sample

This project is the reason I'm applying to this fellowship rather than a general AI-development program. Building it surfaced two problems that shifted how I think about AI systems — from "how do I make AI do more" toward "how do I make sure AI does the right thing reliably."

1. Credential exposure risk

The initial version called the LLM API directly from the browser. That works in a private, sandboxed environment, but breaks the moment the app is hosted publicly — a client-side API key can be read by anyone who opens developer tools. Rather than ship something that looked functional but leaked a credential, I removed the live call entirely for the public deployment. A production version of this would route requests through an authenticated backend that never exposes the key client-side.

2. Preventing confident wrong answers

The system prompt explicitly constrains the model to only use the aggregated data provided, and to say plainly when a question can't be answered from that data, rather than generating a plausible-sounding but fabricated figure. This matters because a business user asking "which stores are underperforming"

has no way to independently verify the AI's answer against the underlying dataset in the moment. If a model hallucinates a number with the same confident tone as a correct one, the user has no signal to catch it. Constraining a model to admit uncertainty is a small design decision with real consequences if skipped.

Both of these are narrow, project-level examples — but they're the kind of problem I want to understand at a deeper, more general level: how do we build AI systems that fail safely and visibly, rather than failing silently in ways users can't detect.

How it works

1. **Pre-aggregation, not raw data.** The underlying dataset has 6,435 store-week records. Rather than sending that to the model, I summarized it first in Python — per-store averages, monthly trends, holiday effects — so the AI works with clean business metrics instead of raw rows, keeping context small and cheap to run.

2. **A constrained system prompt.** The model is told exactly what role to play (a retail analytics consultant), what data it's allowed to use, and to say plainly when a question falls outside that data.

3. **Client-side calculation, model-side interpretation.** Sorting, totals, and chart values are computed in plain JavaScript, not by the model — the AI is used only for language and interpretation, not arithmetic it could get wrong.

Process and background

My background is in applied data analytics (SQL, Python, Power BI), not formal software engineering. I built this using AI-assisted development — I defined the product requirements, the data-handling approach, and the constraints on the AI's behavior; the AI handled the syntax. I reviewed and tested the output, including validating the embedded dataset for errors before deployment, and diagnosing the credential-exposure issue described above once the app moved from a private to a public environment.